LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 2

Stack and Queue

Oleh:

Putri Deissyta Herliyana **NIM. 2510817320010**

PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
APRIL
2026

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
MODUL 2

Laporan Praktikum Algoritma & Struktur Data Modul 2: Stack and Queue ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Prakitkum ini dikerjakan oleh:

Nama Praktikan : Putri Deissyta Herliyana
NIM : 2510817320010

Menyetujui,	Mengetahui,
Asisten Praktikum	Dosen Penanggung Jawab Praktikum
Muhammad Fauzan Ahsani	Ir. Muhamamd Alkaff, S,Kom, M.Kom, Ph.D
NIM. 2310817310009	NIP. 198606132015041001

DAFTAR ISI

LEMBAR PENGESAHAN ii

DAFTAR ISI iii

DAFTAR GAMBAR iv

DAFTAR TABEL v

SOAL 1: Implementasi Stack 1

A. Source Code 1

B. Output Program 4

C. Pembahasan 4

SOAL 2: Si Palui dan Mesin Hitung Aneh 6

A. Source Code 8

B. Output Program 10

C. Pembahasan 10

SOAL 3: Implementasi Queue 13

A. Source Code 13

B. Output Program 16

C. Pembahasan 17

SOAL 4: Si Palui dan Deretan Angka 21

A. Source Code 22

B. Output Program 24

C. Pembahasan 24

GITHUB 27

DAFTAR GAMBAR

Gambar 1.Screenshot Hasil Jawaban Soal 2 10
Gambar 2.Screenshot Hasil Jawaban Soal 4 24

DAFTAR TABEL

Table 1. Source Code stack.h 1
Table 2. Source Code stack.cpp 2
Table 3. Source Code prob1.cpp 8
Table 4. Source Code queue.h 13
Table 5. Source Code queue.cpp 14
Table 6.Source Code prob2.cpp 22

SOAL 1: Implementasi Stack

Pada file stack.h yang diberikan, implementasikan seluruh fungsi yang dideklarasikan ke dalam file stack.cpp. Untuk menangani kondisi tidak valid (invalid case), gunakan mekanisme exception handling dengan throw. Jelaskan alur algoritma dari setiap fungsi yang diimplementasikan, serta berikan justifikasi atau pembuktian singkat mengenai kebenaran algoritma tersebut. Selain itu, lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi yang ada. (Gunakan Big-O Notation).

A. Source Code

Table 1. Source Code stack.h

```
#ifndef STACK_H
#define STACK_H

#define MAX 100

struct Stack {
    int data[MAX];
    int* top;
};

void init(Stack* s);
bool isEmpty(const Stack* s);
bool isFull(const Stack* s);
void push(Stack* s, int value);
void pop(Stack* s);
int peek(const Stack* s);

#endif
```

Table 2. Source Code stack.cpp

```
#include "stack.h"
#include <stdexcept>

using namespace std;

void init(Stack* s) {
    s->top = nullptr;
}

bool isEmpty(const Stack* s) {
    return s->top == nullptr;
}

bool isFull(const Stack* s) {
    return s->top == &s->data[MAX - 1];
}

void push(Stack* s, int value) {
    if (isFull(s)) {
        throw overflow_error("Stack penuh");
    }

    if (isEmpty(s)) {
        s->top = &s->data[0];
    } else {
        s->top++;
    }

```

```
    *(s->top) = value;
}

void pop(Stack* s) {
    if (isEmpty(s)) {
        throw underflow_error("Stack kosong");
    }

    if (s->top == &s->data[0]) {
        s->top = nullptr;
    } else {
        s->top--;
    }
}

int peek(const Stack* s) {
    if (isEmpty(s)) {
        throw underflow_error("Stack kosong");
    }

    return *(s->top);
}
```

B. Output Program

Soal ini tidak menghasilkan output secara langsung karena hanya mengimplementasikan fungsi-fungsi pada struktur data stack. Pengujian dilakukan pada soal berikutnya yang menggunakan stack tersebut.

C. Pembahasan

- Stack.h

Pada baris [1] hingga [3], digunakan preprocessor directive `#ifndef`, `#define`, dan `#endif` untuk mencegah file header di-include lebih dari satu kali.

Pada baris [5], didefinisikan konstanta `MAX` yang menunjukkan kapasitas maksimum stack, yaitu sebanyak 100 elemen.

Pada baris [7] hingga [10], didefinisikan struct `Stack` yang terdiri dari array `data[MAX]` untuk menyimpan elemen dan pointer `top` untuk menunjuk elemen teratas.

Pada baris [12] hingga [18], terdapat deklarasi fungsi-fungsi yang akan digunakan dalam implementasi stack, yaitu `init`, `isEmpty`, `isFull`, `push`, `pop`, dan `peek`. Fungsi-fungsi ini nantinya diimplementasikan pada file `stack.cpp`.

- Stack.cpp

Pada baris [1], `#include "stack.h"` digunakan untuk memanggil file header yang berisi struktur data dan deklarasi fungsi stack. Pada baris [2], `#include <stdexcept>` digunakan untuk menangani kondisi error menggunakan exception seperti `overflow_error` dan `underflow_error`.

Pada baris [6] hingga [8], terdapat fungsi `init()` yang digunakan untuk menginisialisasi stack dalam keadaan kosong dengan cara mengatur pointer `top` menjadi `nullptr`.

Pada baris [10] hingga [12], fungsi `isEmpty()` digunakan untuk mengecek apakah stack kosong. Fungsi ini akan mengembalikan nilai `true` jika `top` bernilai `nullptr`, dan `false` jika stack berisi elemen.

Pada baris [14] hingga [16], fungsi `isFull()` digunakan untuk mengecek apakah stack sudah penuh. Stack dianggap penuh jika pointer `top` sudah menunjuk ke elemen terakhir array, yaitu `data[MAX - 1]`.

Pada baris [18] hingga [29], fungsi `push()` digunakan untuk menambahkan elemen ke dalam stack. Pada baris [19] dilakukan pengecekan apakah stack penuh menggunakan `isFull()`. Jika penuh, maka akan melempar error `overflow_error`. Pada baris [23] hingga [27], dilakukan pengecekan apakah stack kosong. Jika kosong, pointer `top` diarahkan ke elemen pertama array. Jika tidak, pointer `top` digeser ke posisi berikutnya. Pada baris [29], nilai disimpan pada posisi yang ditunjuk oleh `top`.

Pada baris [31] hingga [41], fungsi `pop()` digunakan untuk menghapus elemen teratas stack. Pada baris [32] dilakukan pengecekan apakah stack kosong menggunakan `isEmpty()`. Jika kosong, akan muncul error `underflow_error`. Pada baris [36] hingga [40], jika hanya terdapat satu elemen, maka setelah dihapus stack akan kembali kosong dan `top` diatur menjadi `nullptr`. Jika lebih dari satu elemen, pointer `top` cukup digeser ke belakang.

Pada baris [43] hingga [49], fungsi `peek()` digunakan untuk mengambil nilai elemen teratas tanpa menghapusnya. Jika stack kosong, akan menghasilkan error `underflow_error`. Jika tidak, nilai pada posisi `top` akan dikembalikan.

SOAL 2: Si Palui dan Mesin Hitung Aneh

Time Limit: 1.0 s
Memory Limit: 64 MB

Si Palui menemukan sebuah mesin hitung tua di gudang. Mesin tersebut memiliki cara kerja yang tidak biasa. Alih-alih menggunakan format perhitungan seperti yang biasa kita kenal, mesin ini membaca input berupa deretan simbol yang dipisahkan oleh spasi.

- Setiap simbol bisa berupa:
- sebuah bilangan bulat, atau
- sebuah operator (+, -, *, /)

Cara kerja mesin adalah sebagai berikut:

- Mesin membaca simbol dari kiri ke kanan.
- Jika simbol adalah bilangan, maka bilangan tersebut disimpan ke dalam memori mesin.
- Jika simbol adalah operator, maka mesin akan mengambil dua nilai yang paling baru dimasukkan ke dalam memori yang disimpan, melakukan operasi sesuai operator tersebut, lalu menyimpan kembali hasilnya ke dalam memori.

Setelah seluruh simbol diproses, akan tersisa satu nilai di dalam memori. Nilai inilah yang ditampilkan sebagai hasil akhir.

Sebagai asisten Si Palui, tugas Anda adalah membantu menghitung hasil dari mesin tersebut.

Batasan

- $1 \leq n \leq 100$
- $-1000 \leq Ai \leq 1000$

Keterangan:

- n adalah jumlah simbol dalam ekspresi
- Ai adalah nilai bilangan pada input
- Simbol yang valid hanya:

  - bilangan bulat
  - operator: +, -, *, /

Format Input

```
n

A1 A2 ... An
```

Keterangan:

- Baris pertama n menyatakan jumlah atau panjang simbol
- Baris kedua berisi simbol A sebanyak n buah

Format Output

Sebuah bilangan hasil dari kalkulator Si Palui

Contoh Kasus

Input	**Output**
3 2 3 +	5
9 5 1 2 + 4 * + 3 -	14

5 10 2 / 3 *	15

A. Source Code

Table 3. Source Code prob1.cpp

```
#include <iostream>
#include <string>
#include "stack.h"

using namespace std;

bool isOperator(const string& token) {
    return token == "+" || token == "-" || token == "*" || token == "/";
}

int main() {
    Stack s;
    init(&s);

    int n;
    cin >> n;

    for (int i = 0; i < n; i++) {
        string token;
        cin >> token;

        if (!isOperator(token)) {
            push(&s, stoi(token));
```

```
        } else {
            int b = peek(&s); pop(&s);
            int a = peek(&s); pop(&s);

            int hasil;

            if (token == "+") hasil = a + b;
            else if (token == "-") hasil = a - b;
            else if (token == "*") hasil = a * b;
            else hasil = a / b;

            push(&s, hasil);
        }
    }

    cout << peek(&s) << endl;

    return 0;
}
```

B. Output Program

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\USER\OneDrive\Documents\GitHub\module-2-stack-and-queue-deissytaputri-source> g++ prob1.cpp stack.cpp -o prob1
>> .\prob1
3
2 3 +
5
PS C:\Users\USER\OneDrive\Documents\GitHub\module-2-stack-and-queue-deissytaputri-source> g++ prob1.cpp stack.cpp -o prob1
>> .\prob1
9
5 1 2 + 4 * + 3 -
14
PS C:\Users\USER\OneDrive\Documents\GitHub\module-2-stack-and-queue-deissytaputri-source> g++ prob1.cpp stack.cpp -o prob1
>> .\prob1
5
10 2 / 3 *
15
PS C:\Users\USER\OneDrive\Documents\GitHub\module-2-stack-and-queue-deissytaputri-source>
```

Gambar 1.Screenshot Hasil Jawaban Soal 2

C. Pembahasan

Pada baris [1], `#include <iostream>` digunakan untuk mengimpor library input dan output agar program dapat menggunakan cin dan cout. Pada baris [2], `#include <string>` digunakan karena pada program ini setiap simbol input dibaca dalam bentuk string. Pada baris [3], `#include "stack.h"` digunakan untuk memanggil file header yang berisi struktur data stack beserta fungsi-fungsi yang akan digunakan.

Pada baris [5], using namespace std; digunakan agar penulisan elemen-elemen dari library standar C++ menjadi lebih singkat tanpa perlu menuliskan `std::` secara berulang.

Pada baris [7] sampai [9], terdapat fungsi `isOperator()` yang digunakan untuk mengecek apakah suatu token merupakan operator. Fungsi ini akan mengembalikan nilai true jika token berisi salah satu operator aritmatika, yaitu `+, -, *,` atau `/`. Jika bukan operator, maka fungsi akan mengembalikan nilai false.

Pada baris [11] sampai [42], terdapat fungsi `main()` sebagai bagian utama program. Pada baris [12], dideklarasikan variabel s bertipe Stack yang digunakan sebagai tempat penyimpanan data selama proses perhitungan. Pada baris [13], fungsi

`init(&s)` dipanggil untuk menginisialisasi stack dalam keadaan kosong sebelum digunakan.

Pada baris [15] dan [16], dideklarasikan variabel n, lalu program membaca input n yang menyatakan jumlah simbol pada ekspresi postfix.

Pada baris [18] sampai [37], digunakan perulangan for untuk membaca dan memproses token sebanyak n kali. Pada baris [19], dideklarasikan variabel token bertipe string, kemudian pada baris [20] nilainya dibaca dari input.

Pada baris [22] sampai [23], dilakukan pengecekan apakah token bukan operator dengan memanggil fungsi `isOperator(token)`. Jika token merupakan bilangan, maka token tersebut diubah ke tipe integer menggunakan `stoi(token)`, lalu dimasukkan ke dalam stack dengan fungsi `push(&s, ...)`.

Pada baris [24] sampai [37], jika token merupakan operator, maka program akan mengambil dua elemen teratas dari stack untuk dilakukan operasi. Pada baris [25], elemen teratas diambil menggunakan `peek(&s)` dan disimpan ke variabel b, kemudian elemen tersebut dihapus dengan `pop(&s)`. Pada baris [26], langkah yang sama dilakukan untuk mengambil elemen berikutnya dan menyimpannya ke variabel a.

Pada baris [28], dideklarasikan variabel hasil untuk menyimpan hasil operasi. Pada baris [30] sampai [33], dilakukan pengecekan jenis operator. Jika operator adalah +, maka dilakukan operasi penjumlahan a + b. Jika operator adalah -, maka dilakukan operasi pengurangan a - b. Jika operator adalah *, maka dilakukan operasi perkalian a * b. Selain itu, jika operator adalah /, maka dilakukan operasi pembagian a / b.

Pada baris [35], hasil operasi dimasukkan kembali ke dalam stack menggunakan push(&s, hasil). Hasil ini nantinya dapat dipakai lagi jika masih ada operator berikutnya.

Pada baris [40], setelah semua token selesai diproses, program menampilkan hasil akhir dengan `cout << peek(&s) << endl;`, yaitu nilai elemen teratas

stack. Pada baris [42], `return 0;` menandakan bahwa program selesai dijalankan dengan normal.

Secara keseluruhan, program ini digunakan untuk menghitung ekspresi postfix dengan memanfaatkan struktur data stack. Jika token berupa bilangan, nilainya dimasukkan ke dalam stack. Jika token berupa operator, dua nilai teratas diambil, dihitung sesuai operator, lalu hasilnya dimasukkan kembali ke dalam stack. Setelah seluruh token diproses, akan tersisa satu nilai di dalam stack yang merupakan hasil akhir perhitungan.

SOAL 3: Implementasi Queue

Pada file `queue.h` yang diberikan, implementasikan seluruh fungsi yang dideklarasikan ke dalam file `queue.cpp`. Untuk menangani kondisi tidak valid (invalid case), gunakan mekanisme exception handling dengan `throw`.

Jelaskan alur algoritma dari setiap fungsi yang diimplementasikan, serta berikan justifikasi atau pembuktian singkat mengenai kebenaran algoritma tersebut. Selain itu, lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi yang ada. (Gunakan Big-O Notation).

A. Source Code

Table 4. Source Code queue.h

```
#ifndef QUEUE_H
#define QUEUE_H

#define MAX 100

struct Queue {
    int data[MAX];
    int* front;
    int* rear;
};

void init(Queue* q);
bool isEmpty(const Queue* q);
bool isFull(const Queue* q);
void enqueue(Queue* q, int value);
void dequeue(Queue* q);
int front(const Queue* q);
```

```
18 int back(const Queue* q);
19
20 #endif
```

Table 5. Source Code queue.cpp

```
 1 #include "queue.h"
 2 #include <stdexcept>
 3
 4 using namespace std;
 5
 6 void init(Queue* q) {
 7     q->front = nullptr;
 8     q->rear = nullptr;
 9 }
10
11 bool isEmpty(const Queue* q) {
12     return q->front == nullptr;
13 }
14
15 bool isFull(const Queue* q) {
16     return !isEmpty(q) && q->front == &q->data[0] &&
   q->rear == &q->data[MAX - 1];
17 }
18
19 void enqueue(Queue* q, int value) {
20     if (isEmpty(q)) {
21         q->front = &q->data[0];
22         q->rear = &q->data[0];
23         *q->rear = value;
24         return;
```

```
    }

    if (q->rear == &q->data[MAX - 1]) {
        if (q->front == &q->data[0]) {
            throw overflow_error("Queue penuh");
        }

        int* dest = &q->data[0];
        int* src = q->front;

        while (src <= q->rear) {
            *dest = *src;
            dest++;
            src++;
        }

        int count = q->rear - q->front + 1;
        q->front = &q->data[0];
        q->rear = &q->data[count - 1];
    }

    q->rear++;
    *q->rear = value;
}

void dequeue(Queue* q) {
    if (isEmpty(q)) {
        throw underflow_error("Queue kosong");
    }

```

```
    if (q->front == q->rear) {
        q->front = nullptr;
        q->rear = nullptr;
    } else {
        q->front++;
    }
}

int front(const Queue* q) {
    if (isEmpty(q)) {
        throw underflow_error("Queue kosong");
    }

    return *q->front;
}

int back(const Queue* q) {
    if (isEmpty(q)) {
        throw underflow_error("Queue kosong");
    }

    return *q->rear;
}
```

B. Output Program

Tidak terdapat output program pada soal ini karena hanya berisi implementasi fungsi struktur data queue tanpa program utama (main).

C. Pembahasan

- queue.h

Pada file ini didefinisikan struktur Queue yang terdiri dari array `data[MAX]` untuk menyimpan elemen serta pointer front dan rear untuk menunjuk elemen paling depan dan paling belakang.

Selain itu, pada file ini juga terdapat deklarasi fungsi-fungsi yang digunakan dalam queue, yaitu `init, isEmpty, isFull, enqueue, dequeue, front,` dan `back`. Fungsi-fungsi ini nantinya diimplementasikan pada file queue.cpp.

- queue.cpp

Pada baris [1], `#include "queue.h"` digunakan untuk memanggil file header yang berisi deklarasi struktur `Queue`, konstanta `MAX`, serta prototype fungsi-fungsi queue yang akan diimplementasikan pada file ini. Dengan adanya header tersebut, program dapat mengenali bentuk struktur data dan fungsi yang digunakan.

Pada baris [2], `#include <stdexcept>` dipakai agar program dapat menggunakan mekanisme penanganan error berupa exception, seperti `overflow_error` dan `underflow_error`. Exception ini berguna untuk menangani kondisi queue penuh atau kosong secara lebih aman.

Pada baris [3], `using namespace std;` digunakan supaya penulisan elemen dari standard library tidak perlu diawali dengan `std::`. Dengan begitu, penulisan kode menjadi lebih ringkas dan mudah dibaca.

Pada baris [5]–[8], fungsi `init()` berfungsi untuk menginisialisasi queue agar berada pada kondisi kosong. Hal ini dilakukan dengan mengatur pointer `front`

dan `rear` menjadi `nullptr`. Jika kedua pointer masih `nullptr`, berarti belum ada elemen yang tersimpan di dalam queue.

Pada baris [10]–[12], fungsi `isEmpty()` digunakan untuk mengecek apakah queue kosong. Queue dianggap kosong apabila `front` bernilai `nullptr`. Jika `front` tidak `nullptr`, berarti queue masih memiliki elemen.

Pada baris [14]–[16], fungsi `isFull()` digunakan untuk memeriksa apakah queue sudah penuh. Pada implementasi ini, queue dianggap penuh jika queue tidak kosong, `front` berada di elemen pertama array (`data[0]`), dan `rear` sudah berada di elemen terakhir array (`data[MAX - 1]`). Kondisi ini menunjukkan bahwa seluruh kapasitas array sudah terpakai.

Pada baris [18]–[40], fungsi `enqueue()` digunakan untuk menambahkan elemen ke dalam queue. Pertama, pada baris [19]–[24], dicek apakah queue masih kosong. Jika kosong, maka `front` dan `rear` langsung diarahkan ke elemen pertama array, yaitu `data[0]`, lalu nilai disimpan pada posisi tersebut. Setelah itu fungsi langsung selesai dengan `return`.

Selanjutnya, pada baris [25]–[37], program memeriksa apakah `rear` sudah berada di posisi terakhir array. Jika iya, maka ada dua kemungkinan. Pertama, jika `front` juga masih berada di `data[0]`, berarti seluruh array benar-benar penuh sehingga program akan melempar exception `overflow_error` pada baris [27]–[29].

Kemungkinan kedua, walaupun `rear` sudah di ujung array, ternyata di bagian depan masih ada slot kosong karena sebelumnya pernah dilakukan `dequeue()`. Dalam kondisi ini, pada baris [30]–[35], semua elemen yang masih aktif digeser ke depan array. Pointer `dest` digunakan untuk menunjukkan posisi tujuan mulai dari `data[0]`, sedangkan pointer `src` digunakan untuk membaca data mulai dari

posisi `front`. Proses ini dilakukan dengan loop `while` sampai seluruh elemen dari `front` sampai `rear` selesai disalin.

Pada baris [36]–[38], setelah proses pergeseran selesai, jumlah elemen yang tersisa dihitung melalui variabel `count`. Kemudian `front` diatur kembali ke `data[0]`, sedangkan `rear` diatur ke posisi terakhir elemen yang valid, yaitu `data[count - 1]`. Dengan cara ini, ruang kosong di depan array bisa digunakan lagi.

Pada baris [39]–[40], setelah dipastikan masih ada ruang, `rear` digeser satu langkah ke kanan lalu nilai baru dimasukkan ke posisi tersebut. Jadi, elemen baru selalu ditambahkan di bagian belakang queue.

Pada baris [42]–[51], fungsi `dequeue()` digunakan untuk menghapus elemen paling depan. Pada baris [43]–[45], terlebih dahulu dicek apakah queue kosong. Jika kosong, program akan melempar exception `underflow_error` karena tidak ada elemen yang bisa dihapus.

Jika queue hanya memiliki satu elemen, yaitu ketika `front` sama dengan `rear`, maka pada baris [46]–[49] queue dikembalikan ke kondisi kosong dengan mengatur kedua pointer menjadi `nullptr`.

Namun, jika jumlah elemen lebih dari satu, maka pada baris [50] pointer `front` cukup digeser satu posisi ke kanan. Artinya, elemen terdepan dianggap terhapus tanpa perlu menggeser seluruh isi array. Cara ini lebih efisien dibandingkan harus memindahkan semua elemen setiap kali `dequeue()` dilakukan.

Pada baris [53]–[58], fungsi `front()` digunakan untuk mengambil nilai elemen paling depan tanpa menghapusnya. Jika queue kosong, maka program akan

melempar `underflow_error`. Jika tidak kosong, fungsi akan mengembalikan nilai yang ditunjuk oleh pointer `front`.

Pada baris [60]–[65], fungsi `back()` digunakan untuk mengambil nilai elemen paling belakang. Sama seperti fungsi sebelumnya, jika queue kosong maka akan muncul exception `underflow_error`. Jika queue berisi elemen, maka nilai yang ditunjuk oleh pointer `rear` akan dikembalikan.

Secara keseluruhan, implementasi ini sudah menerapkan konsep dasar queue dengan prinsip FIFO (First In First Out), yaitu elemen yang pertama masuk akan menjadi elemen pertama yang keluar. Hal ini terlihat dari proses `enqueue()` yang selalu menambahkan data di bagian belakang, sedangkan `dequeue()` menghapus data dari bagian depan. Selain itu, implementasi ini lebih efisien dibandingkan queue array biasa yang selalu menggeser data saat penghapusan, karena pergeseran hanya dilakukan ketika `rear` sudah mencapai ujung array tetapi masih ada ruang kosong di depan.

SOAL 4: Si Palui dan Deretan Angka

Time Limit: 1.0 s
Memory Limit: 64 MB

Si Palui memiliki sebuah deretan angka yang tersusun rapi. Ia tertarik untuk mengamati sebagian angka secara berurutan dengan jumlah tertentu.

Ia memilih sejumlah k angka yang bersebelahan, lalu menghitung jumlahnya. Setelah itu, ia menggeser pilihannya satu langkah ke kanan dan kembali menghitung jumlah dari angka-angka yang terpilih.

Proses ini dilakukan terus hingga tidak memungkinkan lagi untuk memilih k angka berurutan. Sebagai asistennya, bantulah Si Palui untuk menentukan hasil perhitungan tersebut.

Batasan

- $1 \leq n \leq 100$
- $-1 \leq k \leq n$
- $-1000 \leq A_i \leq 1000$

Keterangan:

- n: jumlah elemen
- k: ukuran jendela
- A_i: elemen array

Format Input

n k

A1 A2 ... An

Keterangan:

- Baris pertama n menyatakan jumlah atau panjang array. Lalu k besar atau jumlah elemen dalam satu kelompok ayng dibentuk

- Baris kedua berisi simbol A sebanyak n buah

Format Output

Cetak hasil tiap masing-masing kelompok dalam satu baris dipisahkan dengan spasi

Contoh Kasus

Input	Output
5 3 1 2 3 4 5	6 9 12
5 5 1 2 3 4 5	15
6 2 4 -1 2 -3 5 1	3 1 -1 2 6

A. Source Code

Table 6.Source Code prob2.cpp

```
#include <iostream>
#include "queue.h"

using namespace std;

int main() {
    Queue q;
    init(&q);

    int n, k;
    cin >> n >> k;

    int sum = 0;
    bool firstOutput = true;

```

```
    for (int i = 0; i < n; i++) {
        int x;
        cin >> x;

        enqueue(&q, x);
        sum += x;

        while (!isEmpty(&q) && (q.rear - q.front + 1) > k) {
            sum -= front(&q);
            dequeue(&q);
        }

        if (!isEmpty(&q) && (q.rear - q.front + 1) == k) {
            if (!firstOutput) {
                cout << " ";
            }
            cout << sum;
            firstOutput = false;
        }
    }

    cout << endl;
    return 0;
}
```

B. Output Program

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\USER\OneDrive\Documents\GitHub\module-2-stack-and-queue-deissytaputri-source> g++ prob2.cpp queue.cpp -o prob2
>> .\prob2
5 3
1 2 3 4 5
6 9 12
PS C:\Users\USER\OneDrive\Documents\GitHub\module-2-stack-and-queue-deissytaputri-source> g++ prob2.cpp queue.cpp -o prob2
>> .\prob2
5 5
1 2 3 4 5
15
PS C:\Users\USER\OneDrive\Documents\GitHub\module-2-stack-and-queue-deissytaputri-source> g++ prob2.cpp queue.cpp -o prob2
>> .\prob2
6 2
4 -1 2 -3 5 1
3 1 -1 2 6
```

Gambar 2.Screenshot Hasil Jawaban Soal 4

C. Pembahasan

Pada baris [1], `#include <iostream>` digunakan untuk mengimpor library input dan output agar program dapat menggunakan `cin` dan `cout`. Pada baris [2], `#include "queue.h"` digunakan untuk memanggil file header yang berisi struktur data queue beserta fungsi-fungsi yang digunakan dalam program.

Pada baris [4], `using namespace std;` digunakan agar penulisan elemen dari library standar C++ menjadi lebih singkat tanpa perlu menuliskan `std::` secara berulang.

Pada baris [6] hingga [34], terdapat fungsi `main()` sebagai bagian utama program. Pada baris [7], dideklarasikan variabel `q` bertipe `Queue` yang digunakan untuk menyimpan elemen-elemen pada jendela perhitungan. Pada baris [8], fungsi `init(&q)` dipanggil untuk menginisialisasi queue dalam keadaan kosong sebelum digunakan.

Pada baris [10] dan [11], dideklarasikan variabel `n` dan `k`, kemudian nilainya dibaca dari input. Variabel `n` menyatakan banyaknya elemen dalam deretan angka, sedangkan k menyatakan ukuran subarray yang akan dijumlahkan.

Pada baris [13], dideklarasikan variabel sum dengan nilai awal `0` yang digunakan untuk menyimpan jumlah elemen dalam jendela yang sedang diproses. Pada

baris [14], dideklarasikan variabel `firstOutput` bertipe bool dengan nilai awal true yang berfungsi untuk mengatur pencetakan spasi agar output tetap rapi.

Pada baris [16] hingga [31], digunakan perulangan `for` untuk membaca elemen sebanyak n kali. Pada baris [17] dan [18], dideklarasikan variabel `x`, kemudian nilainya dibaca dari input.

Pada baris [20], nilai x dimasukkan ke dalam queue menggunakan fungsi `enqueue(&q, x)`. Pada baris [21], nilai x juga langsung ditambahkan ke variabel sum agar jumlah elemen dalam jendela tetap terbarui.

Pada baris [23] hingga [26], digunakan perulangan while untuk memastikan jumlah elemen di dalam queue tidak melebihi k. Kondisi `(q.rear - q.front + 1) > k` digunakan untuk menghitung banyak elemen yang sedang berada dalam `queue`. Jika jumlah elemen lebih dari k, maka nilai elemen paling depan dikurangi dari sum pada baris [24], kemudian elemen tersebut dihapus dari queue menggunakan `dequeue(&q)` pada baris [25]. Dengan cara ini, queue hanya menyimpan elemen-elemen yang termasuk dalam subarray aktif.

Pada baris [28] hingga [33], dilakukan pengecekan apakah ukuran `queue` sudah sama dengan `k`. Jika kondisi tersebut terpenuhi, berarti satu subarray lengkap telah terbentuk dan jumlahnya dapat dicetak. Pada baris [29] hingga [31], jika output yang akan dicetak bukan yang pertama, maka program akan mencetak spasi terlebih dahulu agar setiap hasil terpisah dengan rapi. Pada baris [32], nilai sum dicetak sebagai hasil penjumlahan subarray saat ini. Pada baris [33], `firstOutput` diubah menjadi false agar output berikutnya diawali dengan spasi.

Pada baris [33], setelah seluruh elemen selesai diproses, program mencetak endl agar output diakhiri dengan baris baru. Pada baris [34], `return 0;` menandakan bahwa program selesai dijalankan dengan normal.

Secara keseluruhan, program ini digunakan untuk menghitung jumlah dari setiap subarray berukuran k secara berurutan. Program memanfaatkan queue untuk

menyimpan elemen-elemen dalam jendela saat ini dan variabel sum untuk menyimpan total nilainya. Setiap kali ada elemen baru yang masuk dan ukuran queue melebihi k, elemen paling depan akan dikeluarkan sehingga queue selalu berisi tepat k elemen yang sedang dihitung.

GITHUB

https://github.com/deissytaputri-source/Praktikum-Algoritma-dan-Struktur-Data